GitHub Repository Administration Automation — Project Documentation

Overview

This project automates GitHub repository administration using Shell scripting and the GitHub REST API. It retrieves contributor and collaborator information programmatically, enabling efficient auditing of user access and permissions across repositories — without needing to check the GitHub web UI manually for each project.

Environment:

- Host: EC2 instance (Ubuntu)
- Language: Bash
- Tools: `curl`, `jq`
- API: GitHub REST API (`api.github.com`)
- Authentication: GitHub Personal Access Token (Basic Auth over HTTPS)

Step 1: Reference Script Setup

The base script (`list-users.sh`) was sourced from a public shell-scripting practice repository and adapted for use. It performs a `GET` request against the GitHub Collaborators API endpoint and filters the JSON response using `jq` to show only users with **read (pull) access**.

Core logic:

```bash
```

```bash
#!/bin/bash

API_URL="https://api.github.com"
USERNAME=$username
TOKEN=$token

REPO_OWNER=$1
REPO_NAME=$2

function github_api_get {
    local endpoint="$1"
    local url="${API_URL}/${endpoint}"
    curl -s -u "${USERNAME}:${TOKEN}" "$url"
}

function list_users_with_read_access {
    local endpoint="repos/${REPO_OWNER}/${REPO_NAME}/collaborators"
    collaborators="$(github_api_get "$endpoint" | jq -r '.[] | select(.permissi

    if [[ -z "$collaborators" ]]; then
        echo "No users with read access found for ${REPO_OWNER}/${REPO_NAME}."
    else
        echo "Users with read access to ${REPO_OWNER}/${REPO_NAME}:"
        echo "$collaborators"
    fi
}

echo "Listing users with read access to ${REPO_OWNER}/${REPO_NAME}..."
list_users_with_read_access
```

The script takes two positional arguments — repository owner and repository name — making it reusable across any repo the authenticated token has access to.

Step 2: Environment Setup on EC2

Dependencies installed on the EC2 instance:

bash

```bash
sudo apt update
sudo apt install -y git curl jq
```

GitHub credentials were set as environment variables for the script to consume:

bash

```
export username="<github-username>"
export token="<personal-access-token>"
```

Script made executable:

```bash
chmod 777 list-users.sh
```

Step 3: Script Execution and Verification

The script was tested against a personal repository:

```bash
./list-users.sh farisneendoor Devops_Scripting-
```

Output:

```
Listing users with read access to farisneendoor/Devops_Scripting-...
Users with read access to farisneendoor/Devops_Scripting-:
farisneendoor
```

It was also tested against another repository (`shell-project`) with multiple collaborators:

```
Listing users with read access to devopsjourneyy/shell-project...
Users with read access to devopsjourneyy/shell-project:
shaihanath2412-glitch
farisneendoor
```

This confirmed the script correctly identifies all collaborators with read access, across different repositories and owners, using a single reusable script.

Step 4: Publishing the Script to a Personal Repository

To version-control the script under a personal GitHub account (separate from the source practice repo), the following workflow was used:

1. Clone the target personal repository:

```bash
git clone https://github.com/farisneendoor/Devops_Scripting-
cd Devops_Scripting-
```

2. Copy the script into the repo:

```bash
cp /home/ubuntu/shell-scripting-projects/github-api/list-users.sh .
```

3. Stage, commit, and push:

```bash
git add list-users.sh
git commit -m "Add GitHub API collaborator listing script"
git push origin main
```

Result:

```
1 file changed, 42 insertions(+)
create mode 100755 list-users.sh
...
d25e08d..4dff8d7  main -> main
```

The push completed successfully, confirming the script is now version-controlled in the personal GitHub repository.

Issue encountered: HTTPS authentication prompt

Git prompted for a username and password on push, since GitHub deprecated password-based authentication in 2021.

Fix: The GitHub username was entered as the username, and a **Personal Access Token** was entered in place of the password — this is required for all HTTPS-based git operations (clone, push, pull) going forward.

Note on commit identity

Git generated a default commit author automatically (`Ubuntu <ubuntu@ip-...>`), based on the system's username/hostname. For proper attribution, this should be set explicitly:

```bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Summary

Phase	Status
Script sourced and reviewed	✅ Complete
Dependencies (`git`, `curl`, `jq`) installed on EC2	✅ Complete
GitHub API authentication configured	✅ Complete
Script tested against multiple repos	✅ Complete
Script published to personal GitHub repo	✅ Complete

The result is a reusable Bash utility that queries the GitHub API to list collaborators with read access on any repository, supporting efficient access auditing across multiple projects without manual UI checks.

Security Note

Personal Access Tokens were used for authentication during this project. Tokens should always be treated as sensitive credentials:

- Never commit tokens into script files or version control
- Revoke and rotate a token immediately if it is ever accidentally exposed (e.g. shared in chat, logs, or screenshots)
- Prefer environment variables or a credential manager over hardcoding